

LAPORAN MODUL 8 PRAKTIKUM PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 8

PEWARISAN

A. TUJUAN PRAKTIKUM

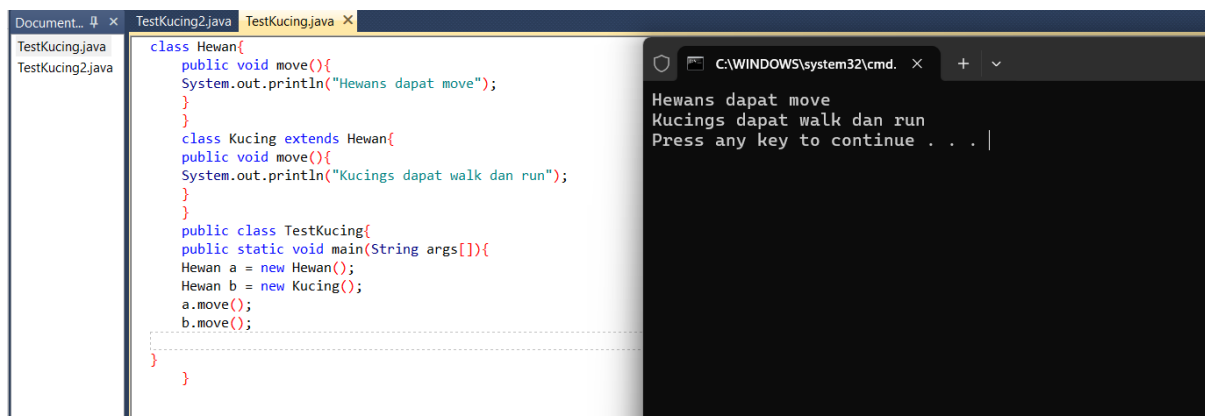
1. Dapat mendefinisikan kelas turunan, Dapat membuat Objek turunan, Dapat mengakses atribut turunan
2. Dapat menerapkan kons-truktor pada sub kelas dalam pewarisan.

B. DASAR TEORI

Dalam bab sebelumnya, kita berbicara tentang kelas-kelas yang super dan sub kelas. Jika kelas mewarisi metode dari kelas super, maka ada kesempatan untuk mengganti metode tersebut asalkan tidak ditandai final. Manfaat utama adalah: kemampuan untuk menentukan perilaku yang khusus untuk jenis subclass yang berarti subclass dapat menerapkan metode kelas induk berdasarkan kebutuhannya. Dalam hal berorientasi objek, utama berarti mengesampingkan fungsi dari metode yang ada

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1



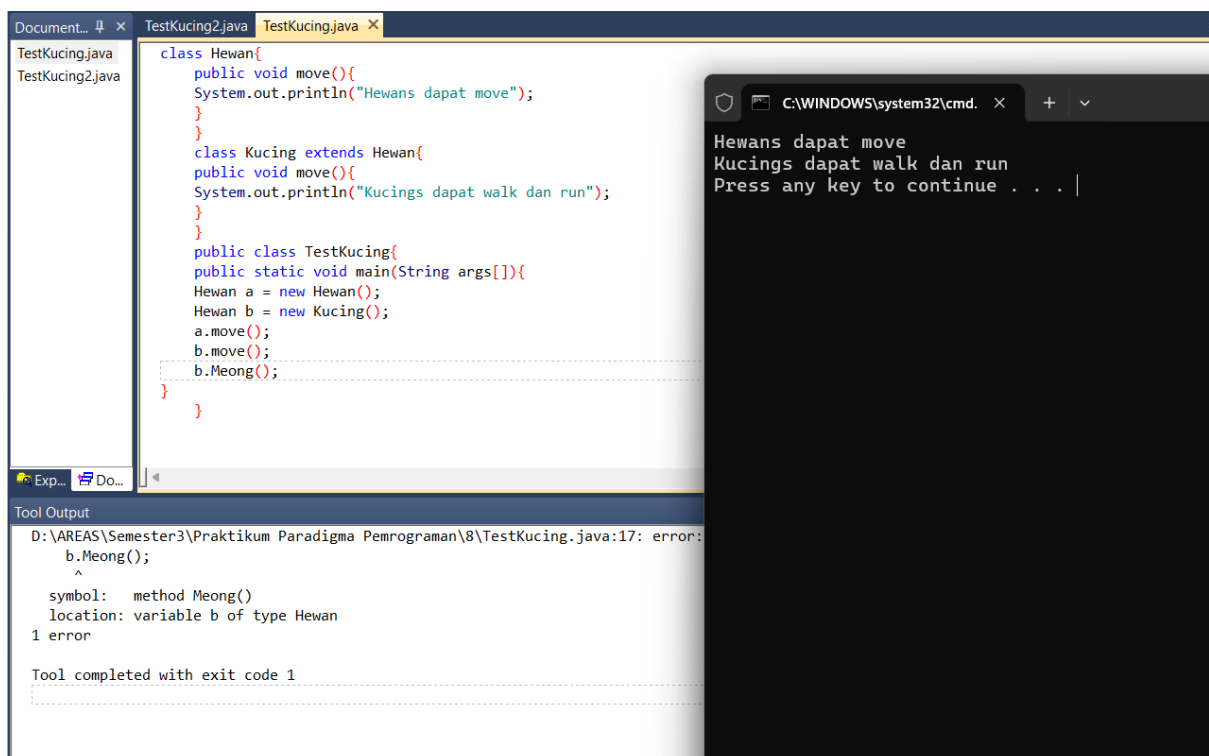
```
class Hewan{
    public void move(){
        System.out.println("Hewans dapat move");
    }
}
class Kucing extends Hewan{
    public void move(){
        System.out.println("Kucings dapat walk dan run");
    }
}
public class TestKucing{
    public static void main(String args[]){
        Hewan a = new Hewan();
        Hewan b = new Kucing();
        a.move();
        b.move();
    }
}
```

C:\WINDOWS\system32\cmd. x + v

Hewans dapat move
Kucings dapat walk dan run
Press any key to continue . . . |

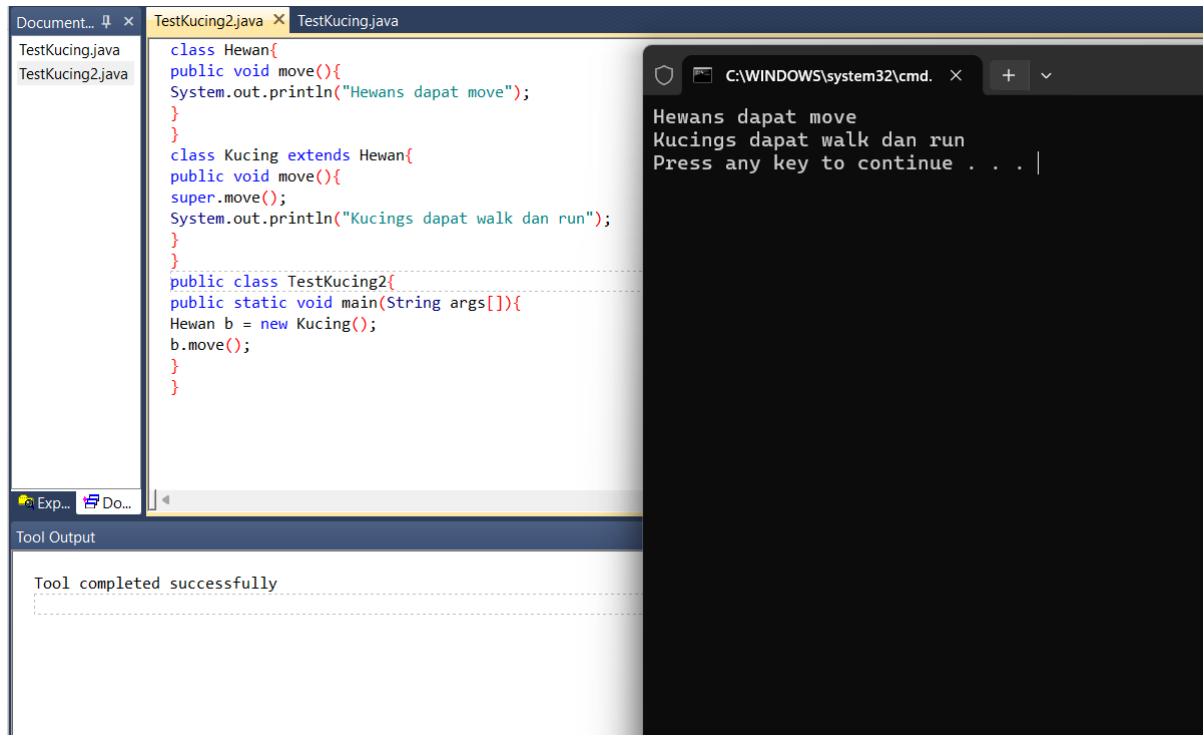
Pembahasan : objek b bertipe Hewan tapi instance-nya Kucing, sehingga method move() yang dipanggil adalah versi override dari Kucing.

Praktik 2



Pembahasan : `b.Meong()`; juga error karena variabel `b` bertipe `Hewan`, sedangkan method `Meong()` hanya ada di `Kucing`. Untuk memanggilnya, harus downcasting: `((Kucing)b).Meong()`; Pemanggilan `Meong()` lewat referensi `Hewan` tidak valid.

Praktik 3



The screenshot shows an IDE with two tabs: `TestKucing2.java` and `TestKucing.java`. The `TestKucing2.java` tab is active, displaying the following Java code:

```
class Hewan{
    public void move(){
        System.out.println("Hewans dapat move");
    }
}
class Kucing extends Hewan{
    public void move(){
        super.move();
        System.out.println("Kucings dapat walk dan run");
    }
}
public class TestKucing2{
    public static void main(String args[]){
        Hewan b = new Kucing();
        b.move();
    }
}
```

Below the code editor is the `Tool Output` panel, which displays the message: `Tool completed successfully`.

To the right of the IDE is a Windows command prompt window titled `C:\WINDOWS\system32\cmd.`. It shows the output of the Java program:

```
Hewans dapat move
Kucings dapat walk dan run
Press any key to continue . . . |
```

Pembahasan : `super.move()` memanggil versi `move()` dari superclass (`Hewan`), lalu ditambah output dari subclass (`Kucing`).

D. PEMBAHASAN TUGAS

```
J Main.java > Main > main(String[])
1 // File: MainHewan.java
2 public class Main {
    Run | Debug
3     public static void main(String[] args) {
4         System.out.println(x: "=== Informasi Hewan Lengkap ===");
5
6         Ciri singa = new Ciri(
7             nama: "Singa",
8             habitat: "Darat",
9             jenis: "Mamalia",
10            makanan: "Daging (Karnivora)",
11            caraBergerak: "Berjalan/Berlari"
12        );
13
14
15        singa.tampilInfo();
16
17        System.out.println(x: "\n=== Informasi Hewan Lain ===");
18        Ciri elang = new Ciri(
19            nama: "Elang",
20            habitat: "Udara/Pohon",
21            jenis: "Aves",
22            makanan: "Daging (Karnivora)",
23            caraBergerak: "Terbang"
24        );
25
26        elang.tampilInfo();
27    }
28 }
```

J Ciri.java > Ciri

```
1 // File: Ciri.java
2 public class Ciri extends Jenis {
3     private String makanan;
4     private String caraBergerak;
5
6     // Constructor
7     public Ciri(String nama, String habitat, String jenis, String makanan,
8         super(nama, habitat, jenis);
9         this.makanan = makanan;
10        this.caraBergerak = caraBergerak;
11    }
12
13    @Override
14    public void tampilInfo() {
15        super.tampilInfo();
16        System.out.println("Makanan : " + makanan);
17        System.out.println("Bergerak : " + caraBergerak);
18    }
19 }
```

```
J Jenis.java >  Jenis >  tampilInfo()
1  // File: Jenis.java
2  public class Jenis extends Hewan {
3      private String jenis;
4
5      // Constructor
6      public Jenis(String nama, String habitat, String jenis) {
7          super(nama, habitat);
8          this.jenis = jenis;
9      }
10
11
12      @Override
13      public void tampilInfo() {
14           super.tampilInfo();
15          System.out.println("Jenis      : " + jenis);
16      }
17  }
```

```
J Hewan.java > Hewan > Hewan(String, String)
1 // File: Hewan.java
2 public class Hewan {
3
4     private String nama;
5     private String habitat;
6
7
8     public Hewan(String nama, String habitat) {
9         this.nama = nama;
10        this.habitat = habitat;
11    }
12
13
14    public void tampilInfo() {
15        System.out.println("Nama Hewan: " + nama);
16        System.out.println("Habitat : " + habitat);
17    }
18 }

PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\9> d:; cd 'd:\AREAS\Semester3\Praktikum
Paradigma Pemrograman\9'; & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsIn
ExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\e27d737e32de
5bcb43438735eaf04445\redhat.java\jdt_ws\9_a0585e65\bin' 'Main'
=== Informasi Hewan Lengkap ===
Nama Hewan: Singa
Habitat : Darat
Jenis : Mamalia
Makanan : Daging (Karnivora)
Bergerak : Berjalan/Berlari

=== Informasi Hewan Lain ===
Nama Hewan: Elang
Habitat : Udara/Pohon
Jenis : Aves
Makanan : Daging (Karnivora)
Bergerak : Terbang
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\9>
```

Pembahasan : Program yang dibuat mengimplementasikan konsep Pewarisan bertingkat (Multilevel Inheritance) dalam pemrograman berorientasi objek menggunakan bahasa Java. Struktur pewarisan ini mengikuti hierarki Hewan → Jenis → Ciri, di mana setiap kelas turunan mewarisi sifat dan perilaku dari kelas induknya

Memanggil Constructor Kelas Induk (super()): Setiap constructor pada kelas turunan (Jenis dan Ciri) harus memanggil constructor dari kelas induk terdekat menggunakan super(...) sebagai baris kode pertama. Misalnya, constructor di kelas Ciri memanggil super(nama, habitat, jenis) untuk meneruskan

data ke constructor kelas Jenis, yang kemudian meneruskannya ke kelas Hewan. Ini memastikan bahwa properti di setiap tingkat hierarki (nama, habitat, jenis, makanan, caraBergerak) terinisialisasi dengan benar.

Mengakses Method Kelas Induk (super.method()): Method tampilInfo() dioverride pada kelas Jenis dan Ciri. Untuk menampilkan informasi dari kelas induk sebelum menambahkan informasi spesifik, method tampilInfo() di kelas turunan menggunakan super.tampilInfo(). Pendekatan ini memastikan output informasi (seperti nama dan habitat dari kelas Hewan) dicetak secara berurutan dan lengkap.

E. KESIMPULAN

Inheritance: pewarisan sifat antar kelas. Overriding & Polymorphism: cara subclass mengubah perilaku superclass dan bagaimana Java memilih method saat runtime.super keyword: cara memanggil method superclass yang ditimpa.Casting: diperlukan untuk memanggil method khusus subclass dari referensi superclass.

F. LAMPIRAN